

CAutomation Project Client Contract

Authoritative platform-level business contract

Status: Authoritative project-level platform contract for CAutomation. This document is intentionally platform-level. Module-specific requirements, screens, entities, endpoints, data fields, schemas, workflows, and acceptance details belong in module-level contracts. When a project-level decision and a module-level detail conflict, the project-level decision controls unless the project-level contract is formally changed.

Review rule applied in this iteration: every section answers the question, "If this were the only information available to the AI about this topic, could it make the correct architectural decision?" Sections have been enriched only where needed to remove platform-level ambiguity.

1. Document Purpose and Decision Boundary

This contract defines the business and product expectations for the complete CAutomation platform. It exists to prevent the AI from treating individual module contracts as isolated applications or from inventing missing platform behavior.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Use this document to decide platform scope, platform concepts, ownership, lifecycle, cross-module rules, business quality requirements, constraints, and platform acceptance.
- Do not use this document to infer module-specific fields, screens, reports, endpoints, table structures, or detailed module workflows.
- If a module contract is silent about a shared platform rule, apply this contract. If this contract is silent about a module detail, the AI must report the gap instead of inventing it.

Topic	Project-level decision	Not decided here
Business scope	The platform manages clients, projects, modules, contracts, AI executions, generated deliverables, validation evidence, review, approval, and export.	The exact domain behavior inside Pipeline Management or User & Client Management.
Authority	Approved project contracts define shared platform behavior for every module.	Module-specific exceptions unless explicitly approved.
Output expectation	An end-to-end web application with platform navigation, authorization, traceability, review, and governance behavior.	Pixel-perfect UI details or implementation-specific naming.

2. Platform Vision

CAutomation is a contract-driven web application platform and AI Engine reference project. The platform must be useful both as a real operational application and as the proving ground for AI-assisted generation.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- The product vision is to make AI generation controlled, auditable, reviewable, and repeatable rather than ad hoc.
- The platform must connect human-authored contracts to normalized inputs, pipeline executions, generated outputs, validation results, and approval decisions.
- The platform must support multiple clients, multiple projects per client, and multiple modules per project without mixing ownership, visibility, or lifecycle state.
- The platform must make uncertainty visible: missing contracts, invalid inputs, failed validations, and unapproved deliverables must be first-class business states.

3. Stakeholders and Responsibilities

The platform must distinguish who owns platform governance, client administration, project decisions, review, technical operation, and AI execution results.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Stakeholder	Business responsibility	Decision authority	Cannot decide
Platform Owner	Owns product direction, platform-wide rules, module model, contract model, acceptance model, and release readiness.	Platform-level behavior and governance.	Client-specific business content.
System Administrator	Maintains operational settings, platform availability, backups, deployment, and technical administration.	Operational administration.	Business approval of deliverables unless assigned separately.
Client Administrator	Manages a client account, client users, client projects, and client visibility.	Client membership and client project administration.	Other clients or platform governance.
Project Owner	Owns project contracts, selected modules, execution readiness, reviews, and final acceptance.	Project lifecycle and deliverable approval.	Security, traceability, or validation rules.
Project Member	Contributes to contracts, reviews, and module-specific project work.	Assigned tasks within project permissions.	Global platform decisions.
Reviewer / Approver	Reviews generated deliverables and validation evidence.	Approval, rejection, or requested rework within assigned scope.	Changing source contracts silently.
AI Engine Operator	Runs permitted pipelines and inspects execution results.	Execution initiation and operational review.	Overriding failed validation.

4. Platform Concepts

These concepts are shared by every module. The AI must use them consistently across navigation, permissions, lifecycle state, reporting, and generated code.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Concept	Platform meaning	Architectural consequence
Client	A business customer or organizational owner of projects, users, contracts, and generated deliverables.	All client-owned data must be isolated by client membership and permissions.
Project	A bounded automation initiative under a client. A project contains project contracts, selected modules, module contracts, executions, reports, and deliverables.	Project context must be present in module workflows and execution history.
Module	A functional domain within a project, such as Pipeline Management or User & Client Management.	Modules share platform rules but own their domain-specific contracts and behavior.
Contract	A human-approved source document used as generation authority.	Contracts require type, scope, ownership, approval status, and traceability.
Pipeline	A platform process made of ordered tasks that transforms approved inputs into outputs, reports, or deliverables.	Execution state and task evidence must be inspectable.
Execution	One run of a pipeline or task against a defined input set.	Every generated artifact must be traceable to an execution.
Deliverable	An output intended for review, approval, export, or application.	Deliverables cannot become accepted without validation evidence.
Validation evidence	Structured proof that inputs or outputs satisfied required checks.	Failed evidence blocks acceptance.

Concept	Platform meaning	Architectural consequence
Approval	A business decision that accepts or rejects a contract, execution output, or deliverable.	Approval must be explicit, attributable, and timestamped.

5. Platform Modules and Scope Separation

CAutomation currently has two reference modules: Pipeline Management and User & Client Management. The project contract defines how modules coexist; module contracts define what each module does.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Pipeline Management is the first reference module and owns pipeline, task, execution, report, and deliverable behavior at module level.
- User & Client Management is the second reference module and owns users, clients, roles, memberships, and client/project administration at module level.
- Shared platform navigation must make modules feel like parts of one application, not separate systems.
- A module may depend on platform concepts such as Client, Project, Role, Permission, Contract, Execution, and Approval, but must not redefine them inconsistently.
- Project-level contracts must never contain the detailed requirements of a module; they define the rules that every module must obey.

6. Contract Model and Source-of-Truth Rules

The platform uses four canonical contract types: two at project level and two at module level. The AI must not add additional contract types unless the project-level model is formally changed.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Approved contracts are authoritative. Draft, obsolete, rejected, or superseded contracts must not be used as generation authority.
- The platform must preserve contract identity, version, scope, status, owner, approval evidence, and relationship to executions.
- A generated result must state which contract versions were used.

Contract	Purpose	Decision boundary
Project Client Contract	Business and product authority for the whole platform.	Platform-wide business rules, concepts, quality attributes, constraints, acceptance, glossary.
Project Engineering Contract	Engineering authority for the whole platform.	Architecture, stack, security, API, data, logging, testing, deployment, AI generation standards.
Software Requirements Specification	Module-level business requirements.	Module-specific capabilities, workflows, actors, acceptance criteria.
Architecture and Technical Specification	Module-level technical requirements.	Module-specific APIs, data structures, UI details, technical behavior.

7. Cross-Module Business Rules

These rules apply across all modules and prevent the AI from making inconsistent business assumptions.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- Client isolation: users may only see clients, projects, contracts, executions, reports, and deliverables they are authorized to access.
- Project context: module actions that affect project-owned data must occur inside a selected project.

- Approval before authority: contracts and deliverables are not authoritative until explicitly approved by an authorized role.
- Validation before acceptance: generated deliverables cannot be accepted when required validation evidence is missing or failed.
- Traceability cannot be optional: every generated artifact, report, and approval must be traceable to source contracts and execution history.
- No hidden state transitions: lifecycle changes such as draft, ready, running, passed, failed, reviewed, approved, rejected, exported, archived, or superseded must be visible.
- Modules must not bypass platform roles, permissions, lifecycle status, or audit requirements.
- Deletion must be controlled. Business-significant records should normally be archived or superseded rather than silently removed.

8. Platform Workflows

The platform must support end-to-end business workflows that connect clients, projects, contracts, modules, generation, review, and approval.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Workflow	Required platform behavior	Completion state
Client onboarding	Create or administer a client, assign client administrators, establish visibility boundaries.	Client active and manageable.
Project setup	Create project under a client, choose modules, attach project contracts, define responsible roles.	Project ready for contract review.
Contract approval	Review uploaded or generated contracts, mark approved/rejected/superseded, preserve version history.	Approved contract set is available for execution.
Execution preparation	Check required contract availability, status, scope, and compatibility before a pipeline run.	Execution can start or readiness gaps are shown.
AI generation	Run authorized pipeline tasks using approved inputs and produce traceable outputs.	Execution passed, failed, or requires review.
Validation and review	Inspect reports, validation evidence, generated artifacts, and unresolved warnings.	Reviewer decision is recorded.
Deliverable approval/export	Approve acceptable deliverables and export packages with evidence.	Deliverable package is accepted and auditable.
Change management	Supersede contracts, rerun affected pipelines, and preserve prior execution evidence.	New baseline replaces old baseline with traceability.

9. Business Quality Requirements

The platform must be judged by business quality, not only by technical correctness.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Quality attribute	Required business meaning	Acceptance signal
Traceability	Users can follow every deliverable back to contracts, pipeline, task, execution, and validation evidence.	No accepted deliverable lacks provenance.
Clarity	Users can understand current status, next action, owner, and blocking issue without reading logs.	Important states are visible in the UI.
Consistency	Shared concepts behave the same across modules.	Same lifecycle and permission rules apply everywhere.
Governance	Approval, rejection, supersession, and export are controlled business actions.	Decision history is attributable.
Recoverability	Failed or invalid executions do not corrupt accepted baselines.	Users can inspect failure and rerun after correction.

Quality attribute	Required business meaning	Acceptance signal
Usability	The platform supports normal business use by non-developer stakeholders.	Core workflows are understandable without code knowledge.
Extensibility	New modules can be added without redefining platform concepts.	Module additions preserve shared rules.

10. Constraints and Assumptions

These constraints keep the AI from inventing conflicting platform requirements.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- CAutomation is the only project name. Legacy names such as CFF, CFFP, CRUD Factory, or AI Factory must not appear in generated platform behavior.
- PDF is the primary supported contract source format at this stage. DOCX may remain supported where already implemented, but project decisions must not assume equal engineering priority.
- The platform must be web-based and end-to-end: frontend, backend API, persistence, authentication/authorization, reports, and export behavior must work together.
- The platform assumes human approval remains required for business-significant contract and deliverable decisions.
- The platform must prefer explicit failure over silent fallback when required information is missing.
- Project-level contracts stay platform-level; module-specific requirements stay in module contracts.

11. Acceptance Criteria

The complete CAutomation platform is acceptable only when platform-wide business behavior is generated coherently and can be validated end-to-end.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

- The application supports client, project, module, contract, execution, report, deliverable, review, approval, and export concepts as one coherent platform.
- Project-level and module-level contract responsibilities remain clearly separated in generated behavior.
- Users cannot access data outside their authorization boundary.
- Pipeline outputs and deliverables are traceable to approved source contracts and execution evidence.
- Invalid or insufficient inputs are surfaced before downstream generation or acceptance.
- Failed validations block acceptance until corrected, waived by an explicit approved policy, or rerun successfully.
- The platform UI exposes status, ownership, blockers, and next actions for key workflows.
- Generated documentation, reports, and exports use the current CAutomation naming and contract model only.

12. Project-Level Glossary

The AI must use these terms consistently in generated platform behavior and must not substitute legacy terminology.

Decision-completeness check: Yes. If this section were the only platform-level information available to the AI about this topic, the AI should make the decision described here and must not invent module-specific details.

Term	Platform definition
CAutomation	The complete contract-driven AI Engine platform and reference web application.
Project-level contract	A contract that defines behavior shared across all modules in a project.
Module-level contract	A contract that defines requirements or technical details for one module only.

Term	Platform definition
Decision-complete	Sufficiently explicit that the AI can make the correct platform-level decision without guessing.
Platform-level	Shared across the entire application and every module.
Module-specific	Owned by a single module and excluded from project-level contracts.
Normalized input	Structured representation of approved source documents used by downstream tasks.
Validation evidence	Structured proof that a contract, input, execution, or deliverable passed required checks.
Deliverable package	A set of generated outputs plus reports and evidence intended for review, approval, or export.
Superseded	No longer current but preserved for traceability and history.